Guia De Desenvolvimento
Rest
Spring MVC
Spring Boot
Git
2026-2
Parte 2

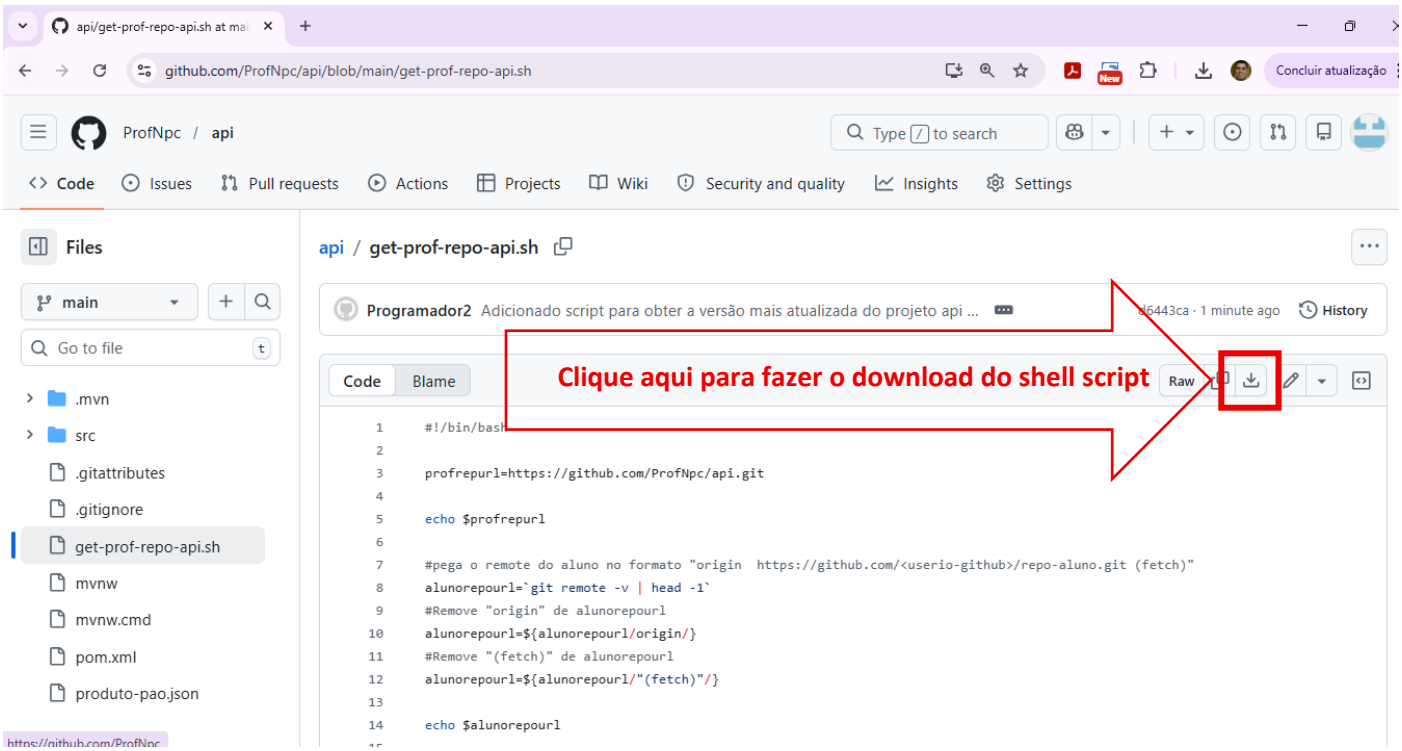
Introdução

Neste documento teremos o passo-a-passo para a construção de uma aplicação web/rest utilizando as ferramentas Spring MVC, Spring Boot e Git.

Este guia se destina a dar suporte as aulas de LIP1 do segundo módulo do curso de informática noturno do ITB Belval, portanto, algumas explicações mais detalhadas serão omitidas pois serão apresentadas pelo professor durante a aula.

O projeto desenvolvido aqui funciona como referência para a criação do *BackEnd* dos projetos de TCC. A cada aula nós avançaremos com o desenvolvimento desse projeto, salvando seu progresso no repositório público https://github.com/ProfNpc/api.

Caso você venha a perder alguma aula, poderá obter a versão mais atualizada feita pelo professor e seguir acompanhando as aulas de onde a turma parou. Para isto basta colocar o *shell script* https://github.com/ProfNpc/api/blob/main/get-prof-repo-api.sh na raiz do projeto api e executá-lo em seguida.



Parte 2

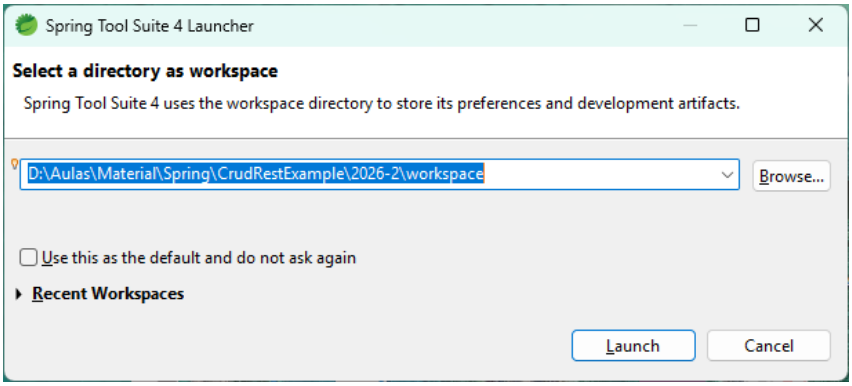
Como representar coisas do mundo real dentro do computador? JavaBean's!

Na maioria das vezes, para criarmos um Controller para nossa API, que é o elemento responsável por responder as requisições HTTP do cliente, faremos esse Controller manipular uma coleção de uma certa entidade. Muitas vezes essa entidade se confunde com as "entidades" que definimos em nosso modelo conceitual do banco de dados e, por consequência, com as tabelas produzidas para implementar aquele modelo conceitual. Aqui nós criaremos um **JavaBean** com os atributos da entidade **Produto**, considerando que para ela há apenas uma tabela com a seguinte estrutura.

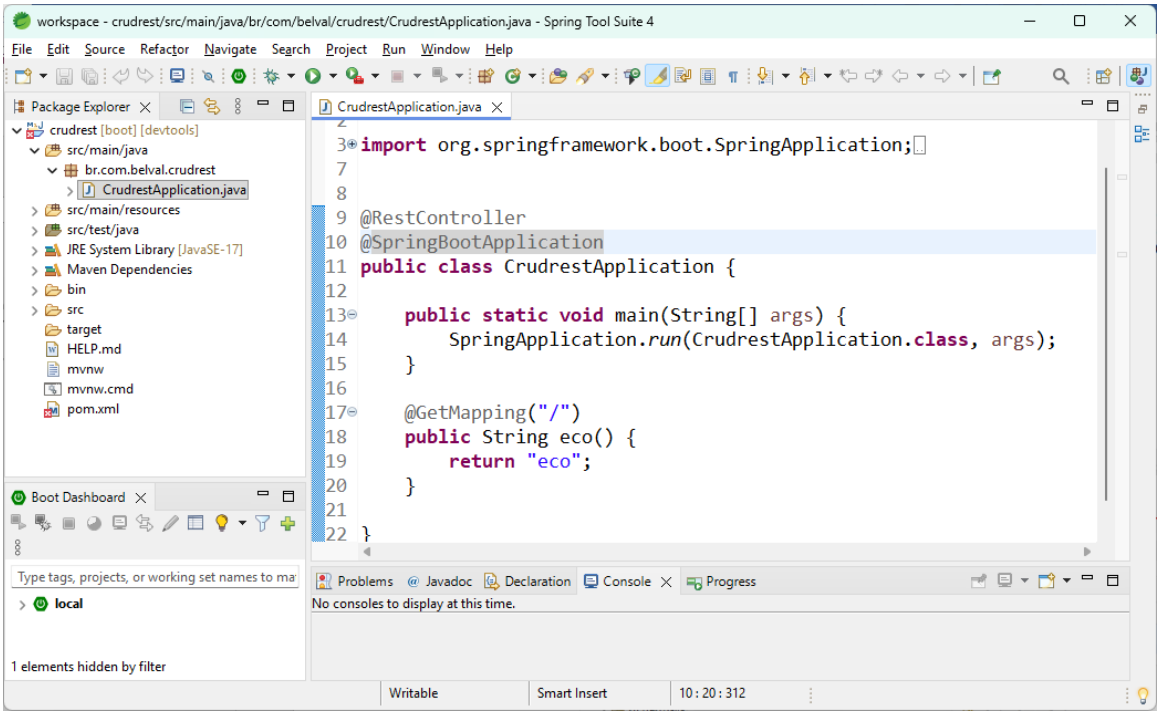
id_produto	nome	descricao	preco	data_criacao
1	Pão	Pão Francês	1.20	01/01/2026 09:58:12
2	Mortadela	Mortadela Sadia	3.50	01/01/2026 10:03:42
etc..				

Criando a classe Produto para criar representações de produtos dentro do computador

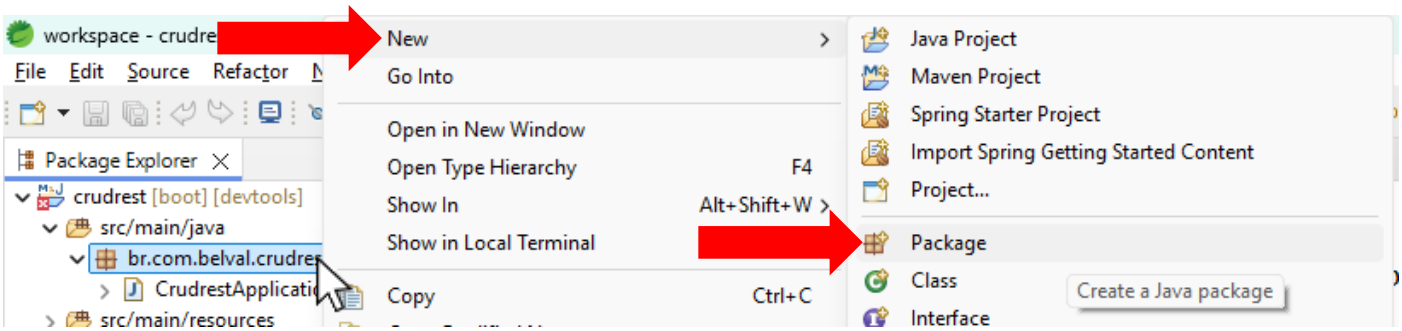
1 – Abra seu workspace que contém o projeto "crudrest", lembre-se que ao abrir o STS, na primeira tela, devemos informar o caminho da pasta "workspace".



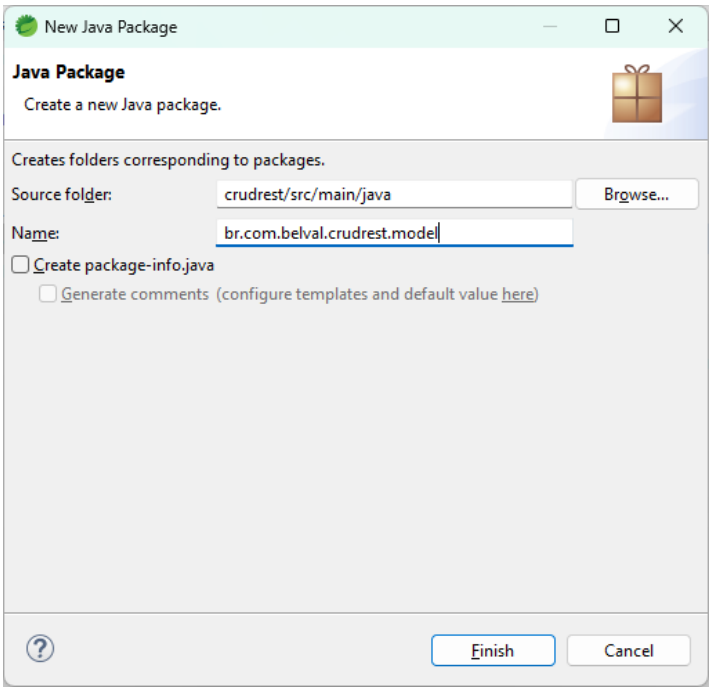
2 – No painel "Package Explorer", encontre o pacote raiz de sua aplicação Spring Boot, onde se encontra a classe "principal" desse tipo de projeto, anotada com `@SpringBootApplication`. No nosso caso, o pacote é `br.com.belval.crudrest`.

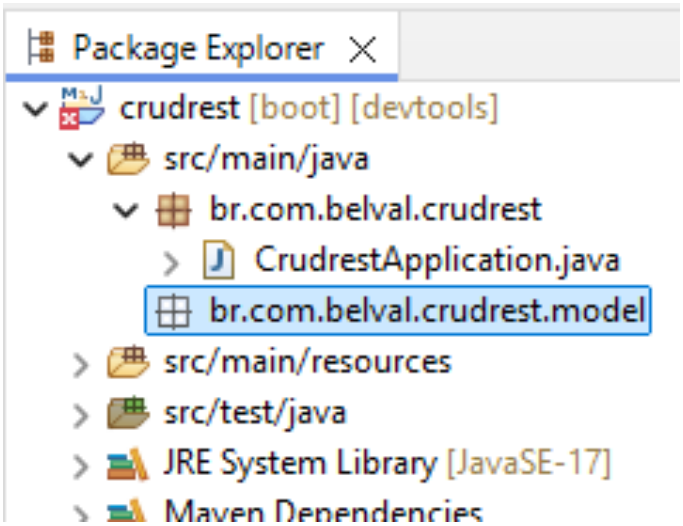


3 – Clique com o botão direito do mouse sobre `br.com.belval.crudrest` >> New >> Package

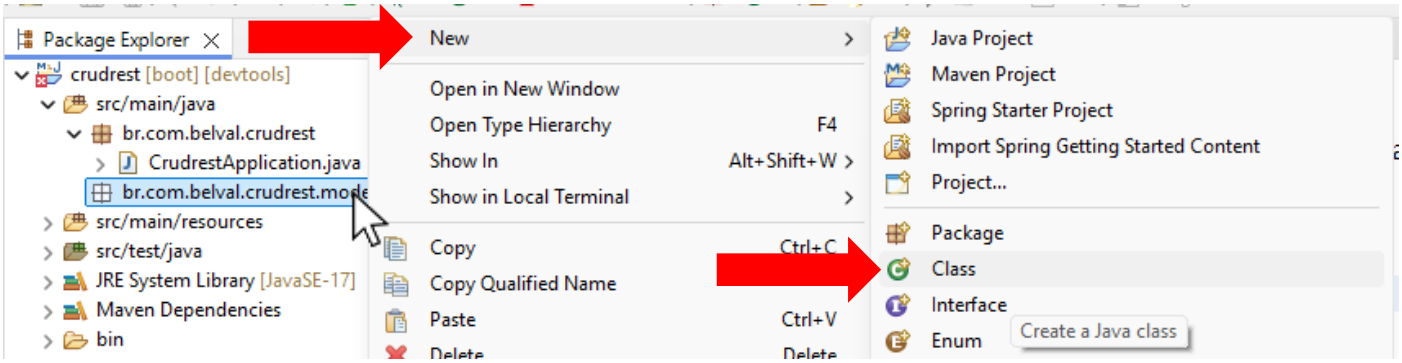


4 – Na janela que se abre, no campo “Name”, adicione ao final do pacote da classe principal “`.model`”, deixando o campo com o valor de `br.com.belval.crudrest.model` e clique em “Finish”.

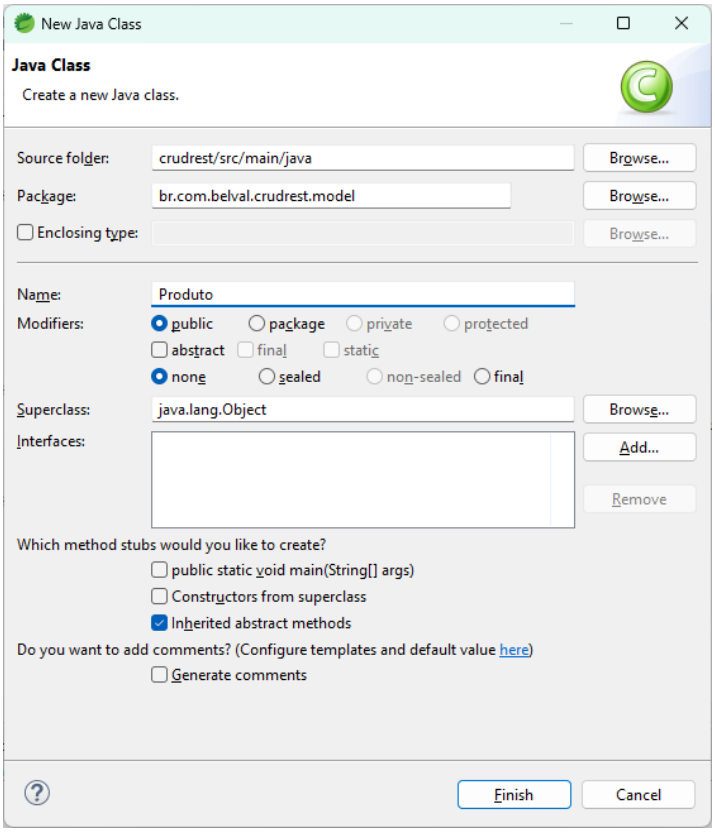


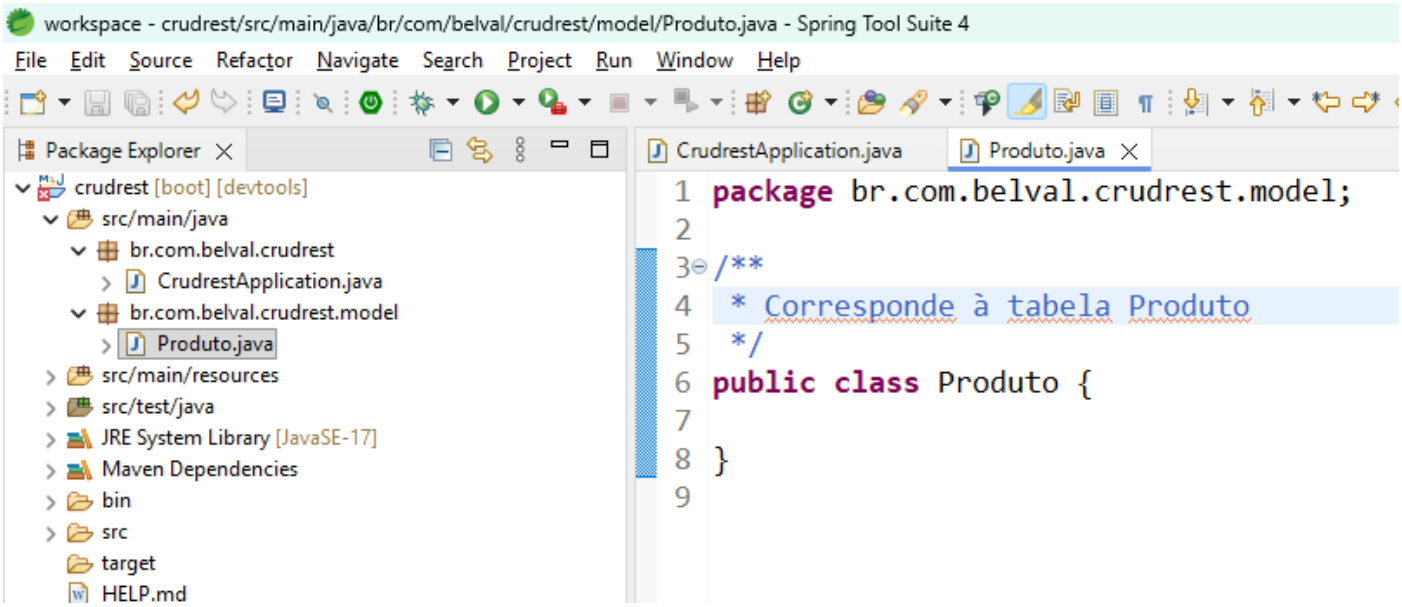


5 – Clique no pacote `br.com.belval.crudrest.model` com o botão direito >> New >> Class



6 – Na janela que se abre, preencha o campo “Name” com “Produto”, em seguida clique em “Finish”.

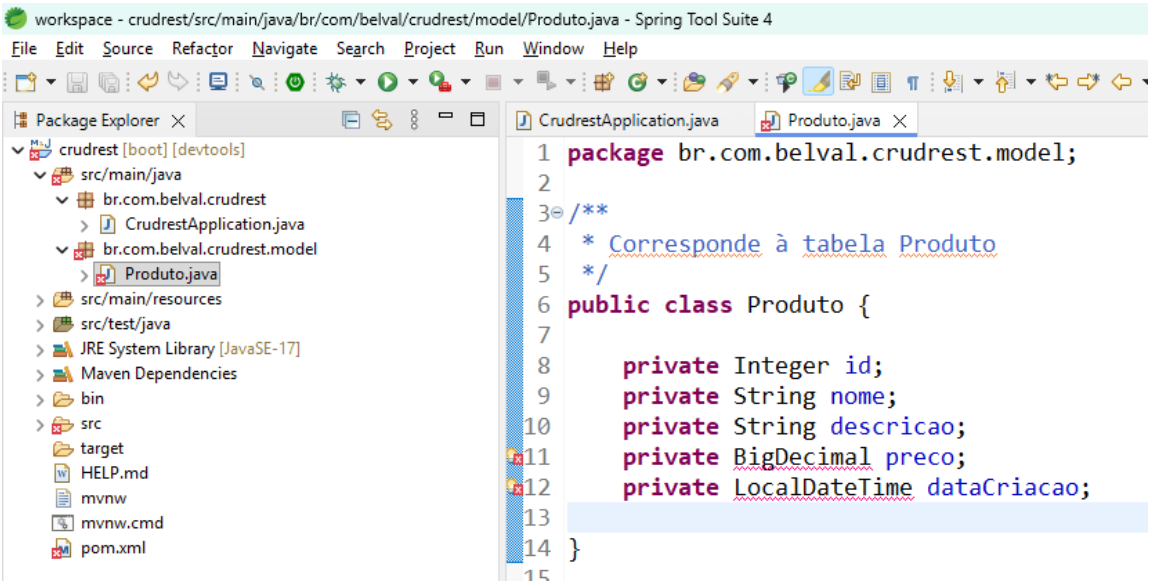




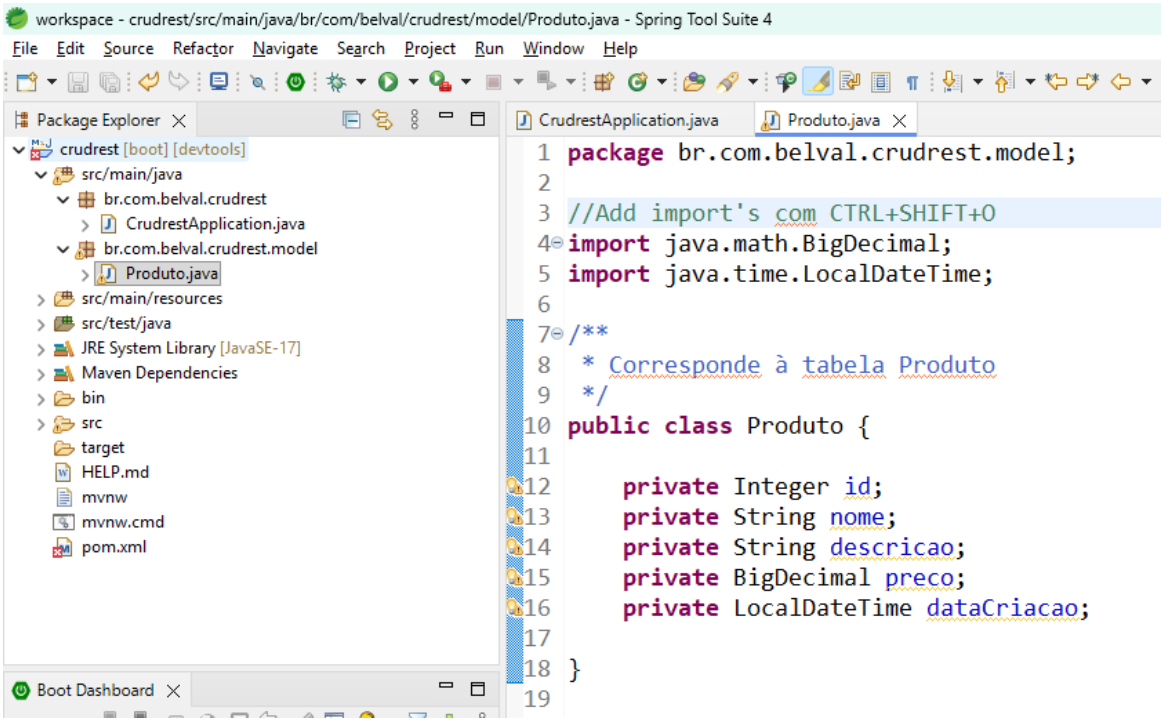
7 – Agora devemos adicionar os atributos dessa classe observando as seguintes condições, dado que utilizaremos esta classe para carregar registros da tabela produto:

- Todos os atributos devem ser privados

- O atributo de `id`, que corresponderá à chave primária da tabela, não deverá ser um tipo primitivo como `int` ou `long` mas sim um tipo *Wrapper*, como `Integer`, `Long`, `BigInteger` ou `UUID`.
- Os campos `nome` e `descricao`, que armazenarão texto, devem ser definidos como `String`
- O campo `preco`, que armazerá valores decimais, mais especificamente, valores monetários, deve ser definido com o tipo `BigDecimal`.
- O campo `dataCriacao`, que armazenará o instante no tempo em que o produto foi criado, deve ser definido como `LocalDateTime` para manter a informação completa de dia, mês, ano, hora, minuto e segundo. Caso precisássemos apenas de dia, mês e ano, poderíamos utilizar `LocalDate`.



8 – Adicione os import's necessários para que a classe deixe de apresentar erro e reconheça as classes BigDecimal e LocalDateTime. Para "importar" a definição das classes BigDecimal e LocalDateTime podemos utilizar o atalho `CTRL + SHIFT + O`. Após adicionar os import's não esqueça de salvar as alterações, você pode utilizar o atalho `CTRL + S`.



9 – Em seguida, precisamos criar um construtor padrão sem parâmetros. Isso será necessário para que o Spring possa instanciar nossa classe `Produto` quando precisar, por exemplo, quando recebermos uma requisição para criar um novo `Produto`, o Spring pegará o corpo da requisição que conterá as informações do produto em **JSON**

(JavaScript Object Notation / Notação de Objeto em JavaScript) e deverá colocá-las nos atributos de um objeto **`Produto`** recém criado pelo Spring como se utilizasse o operador **`new`** em **`Produto`**.

```java
package br.com.belval.crudrest.model;

//Add import's com CTRL+SHIFT+O
import java.math.BigDecimal;
import java.time.LocalDateTime;

/**
 * Corresponde à tabela Produto
 */
public class Produto {

    private Integer id;
    private String nome;
    private String descricao;
    private BigDecimal preco;
    private LocalDateTime dataCriacao;

    /**
     * Método construtor é aquele que:
     * 1 - Tem exatamente o mesmo nome da classe
     * 2 - Não tem tipo de retorno(nem void)
     *
     * Método construtor padrão
     * 1 - Lista de parâmetros vazia
     */
    public Produto() {

    }

}
```

10 – Agora, dado que os atributos são todos privados, precisamos criar os métodos que nos permitam salvar informações nos atributos de uma instância e métodos que nos permitam recuperar essas informações depois, métodos que nos deem acesso aos atributos, esses métodos “acessores" são chamados de *setters* e *getters*. Por exemplo, vamos ver os métodos *getter* e *setter* de para o atributo **`id`**.

```java
    public Produto() {

    }

    //Método que recupera o valor do atributo id
    public Integer getId() {
        return id;
    }

    //Método que define/altera o valor do atributo id
    public void setId(Integer id) {
        this.id = id;
    }


}
```

Esses métodos seguem o padrão JavaBean, isto é, o método que lê o valor do atributo, deve começar com “**`get`**” (ou “**`is`**” caso o atributo seja **`boolean`**) e terminar com o nome do atributo no padrão *camel case*, com a primeira letra

maiúscula, por isso `Id` e não `id`, além disso, esse método `getId` não deve ter parâmetros e deve retornar o valor do atributo como feito pela instrução `return this.id`. O tipo de retorno desse método deve ser igual ao tipo do atributo.

Já o método que permite definir o valor do atributo deve começar com "`set`" e terminar com o nome do atributo, também no padrão *camel case* com a primeira letra maiúscula. Além disso esse método `setId` deve receber um único parâmetro do mesmo tipo do atributo e atribuir o valor recebido por este parâmetro ao atributo, como feito em `this.id = id`, aqui `this`, quer dizer **"a instância à qual o método sendo executado pertence"**, é uma forma de distinguir o `id` parâmetro do `id` atributo. Além disso esse método deve ter como tipo de retorno `void`.

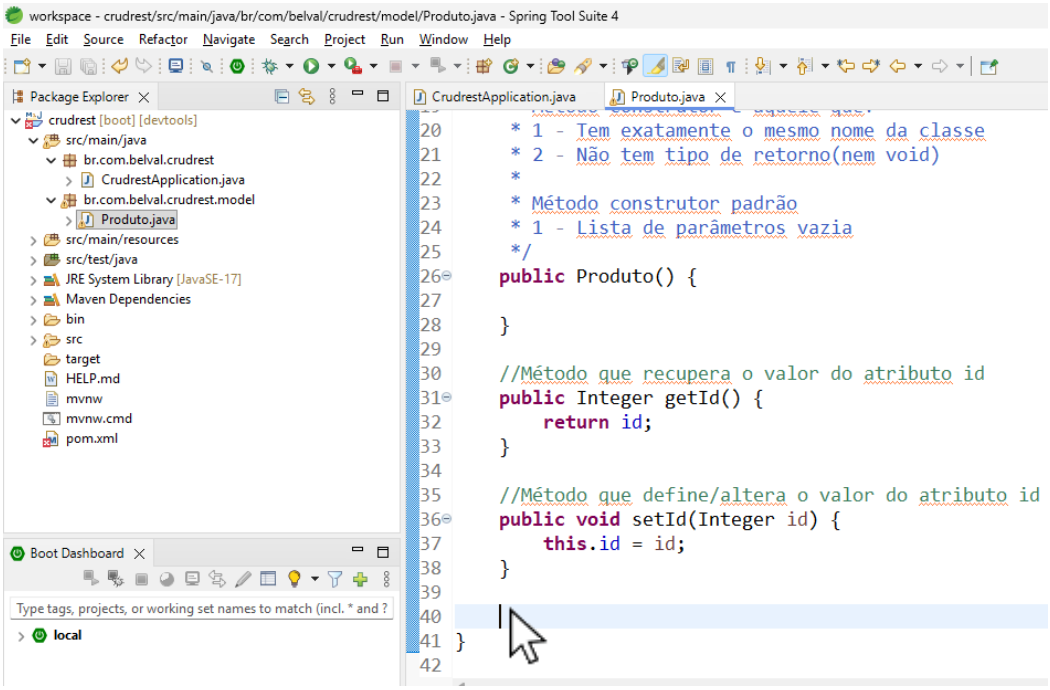
Tipo atributo	Ação	Pref.	Exemplo de atributo	Exemplo de método
`boolean`	Leitura	is	`private boolean ativo;`	`public boolean isAtivo() { return ativo; }`
Tipos não `boolean`	Leitura	get	`private LocalDateTime dataCriacao;`	`public LocalDateTime getDataCriacao() { return dataCriacao; }`
`boolean`	Definição	set	`private boolean ativo;`	`public void setAtivo(boolean ativo) { this.ativo = ativo; }`
Tipos não `boolean`	Definição	set	`private LocalDateTime dataCriacao;`	`public void setDataCriacao(LocalDateTime dataCriacao) { this.dataCriacao = dataCriacao; }`

A partir deste ponto, nossa classe Produto pode ser instanciada em outros pontos do código e podemos salvar e recuperar informações no atributo id.

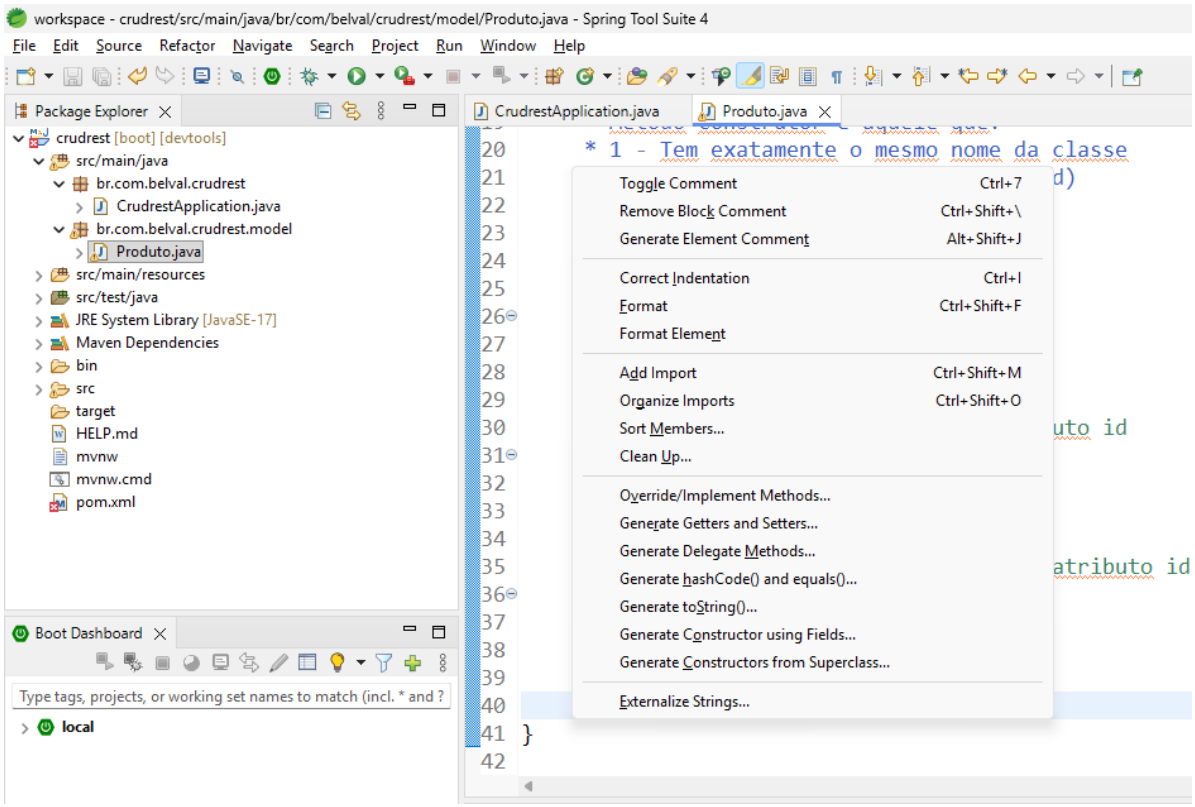
```
public static void main(String[] args) {
    Produto p = new Produto();
    p.setId(223);//atribui o valor para o atributo id dentro do objeto p
    Integer id = p.getId();//Recupera o valor do atributo id do objeto p
    System.out.println("id:" + id);
}
```

11 – Criar os getter's e setter's dos demais atributos utilizando o atalho da IDE.

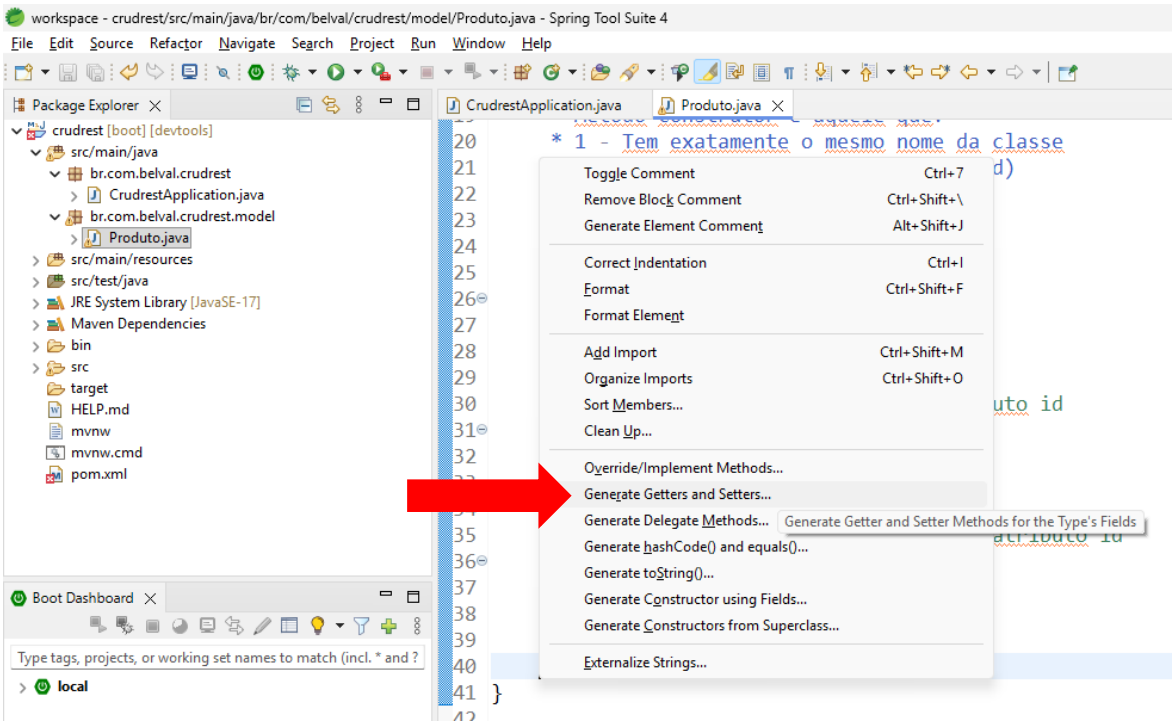
11.1 – Coloque o cursor depois dos getter's e setter's de id.

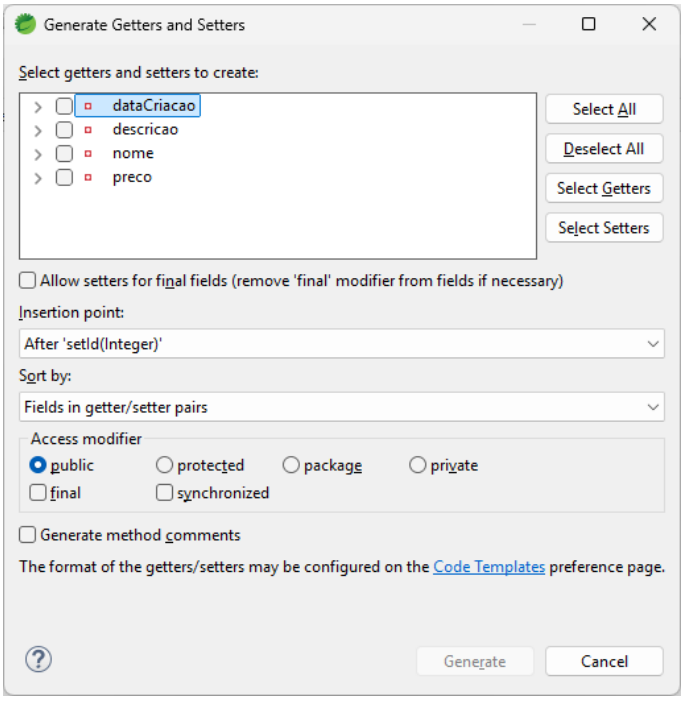


11.2 – Pressione **`ALT + SHIFT + S`**

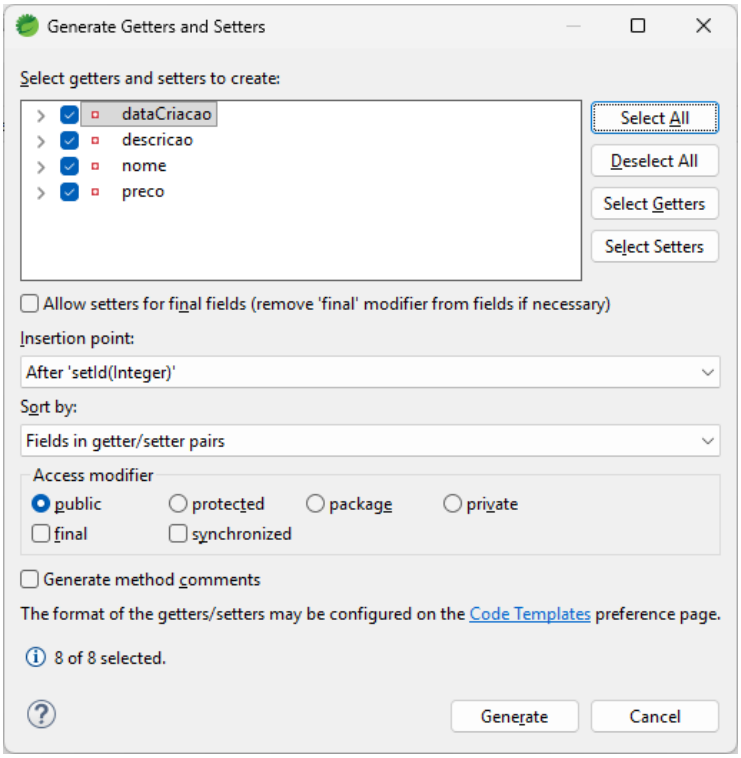


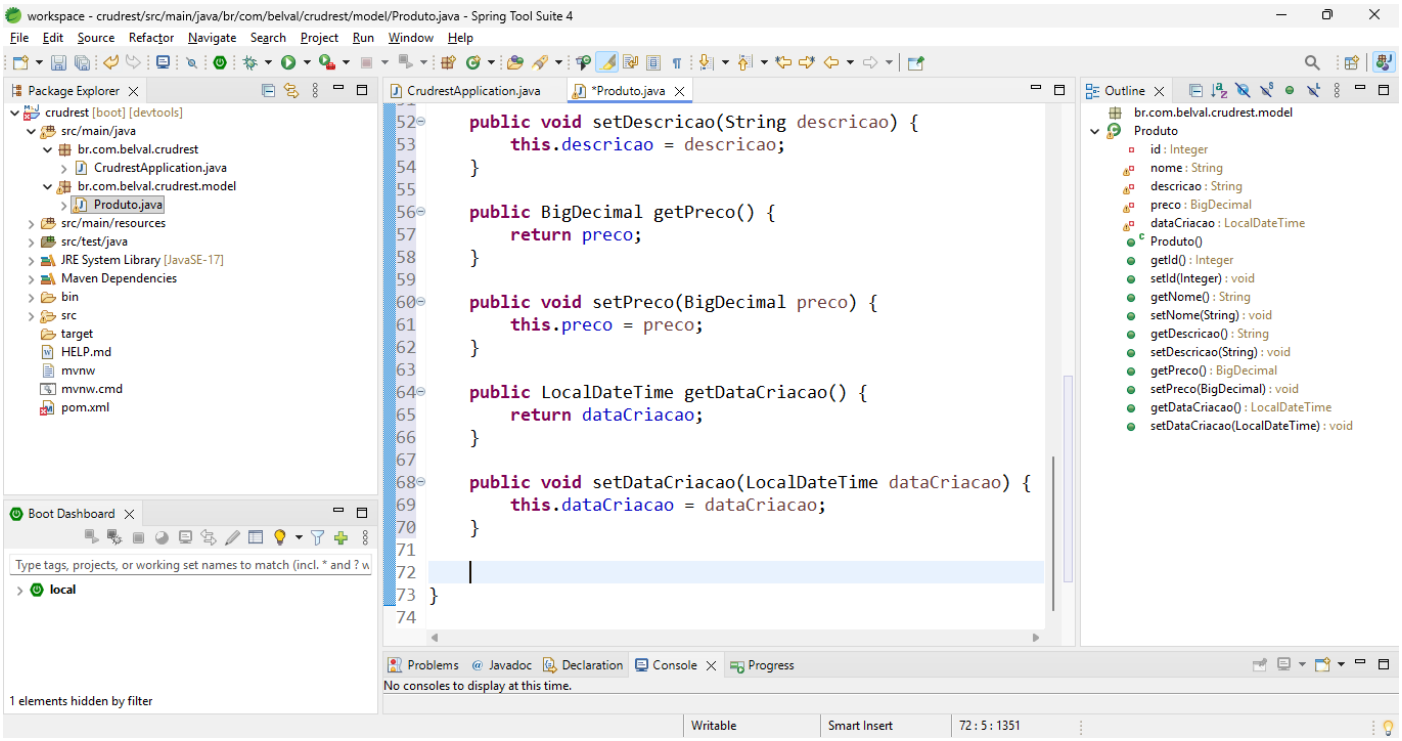
11.3 – Selecione a opção *Generate Getters and Setters*



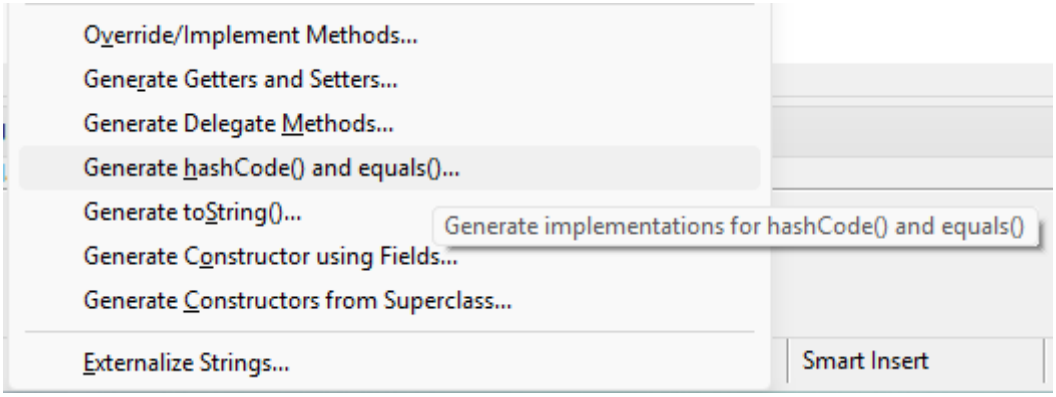


11.4 – Selecione os atributos **`dataCriacao`**, **`descricao`**, **`nome`** e **`preco`** e clique em “Generate”. Com isso todos os getters e setters serão gerados automaticamente.

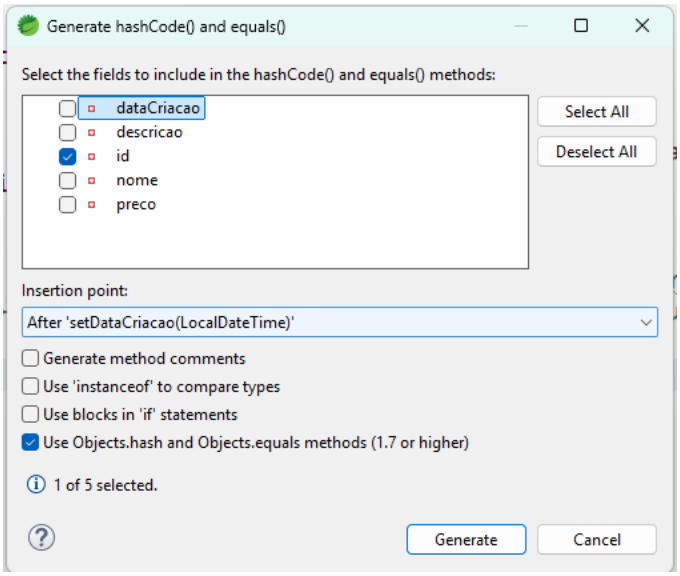




12 – Utilizando os atalhos da IDE, vamos gerar os métodos **`equals`** e **`hashCode`**. Para isso, utilizaremos o atalho **`ALT + SHIFT + S`** >> **`Generate hashCode() and equals()`**.



13 – Selecione apenas o atributo **`id`** e clique em *Generate*



```
99      @Override
100     public int hashCode() {
101         return Objects.hash(id);
102     }
103
104     @Override
105     public boolean equals(Object obj) {
106         if (this == obj)
107             return true;
108         if (obj == null)
109             return false;
110         if (getClass() != obj.getClass())
111             return false;
112         Produto other = (Produto) obj;
113         return Objects.equals(id, other.id);
114     }
115
```

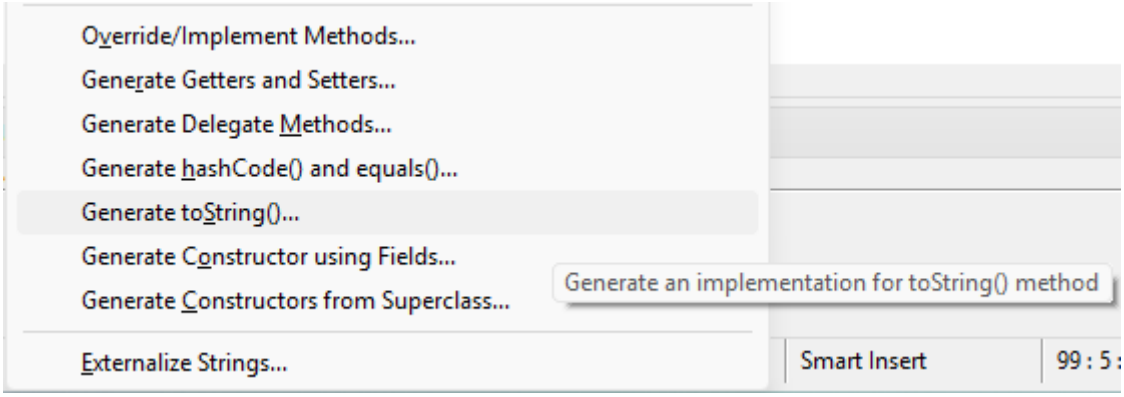
O método **`equals`** serve para dizer se duas instancias diferente são significativamente iguais. Por exemplo, quando formos implementar o método de atualização do **`Produto`** vamos receber um objeto **`Produto`** completo com todos os atributos preenchidos com certos valores. Para que a atualização seja feita, vamos precisar procurar em nossa tabela, se aquele produto já existe para que a atualização seja efetuada. Caso o produto recebido para atualização não exista na tabela, devemos retornar um código de erro indicando que o recurso não foi encontrado, no protocolo **`HTTP`** isso é indicado através do código de retorno **`404 (Not found)`**. Em nossas aplicações, em diferentes situações, semelhantes a essa, precisamos percorrer uma coleção de objetos e se perguntar **"esse objeto A é significativamente igual ao objeto B?"** e refazer essa pergunta para C, D etc. até que um objeto da coleção seja igual ou a lista tenha acabado. Essa pergunta é feita chamando o método **`equals`**. A questão é que devemos escolher com cuidado qual será o critério utilizado pelo método **`equals`** para dizer se dois objetos são iguais ou não. No nosso caso, dois objetos serão iguais se o **`id`** for igual, independente do valor dos outros atributos.

Já o método **`hashCode`** gera um número associado ao estado do objeto, isto é, um número associado aos valores dos atributos do objeto e ele deve ser coerente com os resultados do método **`equals`**.

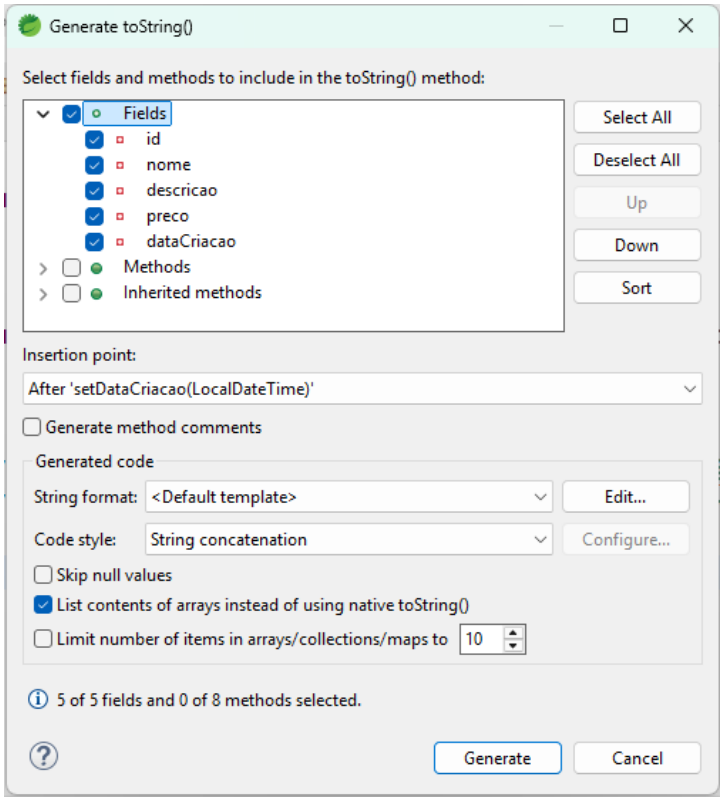
- Se dois objetos são iguais pelo equals, eles DEVEM ter o mesmo hashCode.
- Se dois objetos têm o mesmo hashCode, eles NÃO precisam ser iguais (isso é chamado de colisão).

O método **`hashCode`** serve para distribuir os objetos em várias listas nas chamadas tabelas de espalhamento, chamada em inglês de *hash table*. Resumidamente, o **`hashCode`** serve como o número de CEP das ruas, quando chega uma carta, ela é colocada na caixa relacionada ao CEP que ela tem, dessa forma, quando se precisa procurar uma carta destinada a uma casa, sabendo o CEP, podemos restringir nosso espaço de busca, já que vamos olhar apenas as cartas com determinado CEP em vez de olhar todas as cartas da cidade.

14 – Utilizando os atalhos da IDE, vamos gerar o método **`toString`**. Para isso, utilizaremos o atalho **`ALT + SHIFT + S`** >> **`Generate toString()`**.

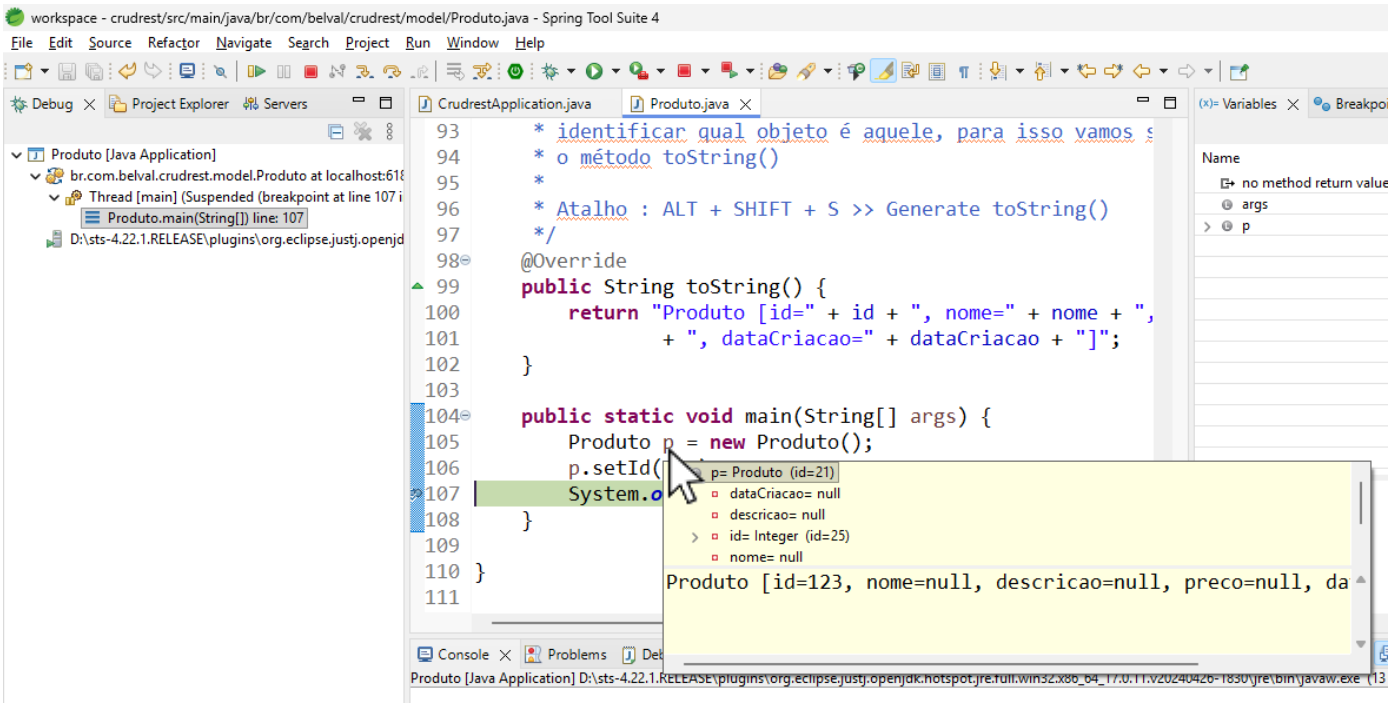


15 – Na tela seguinte, selecione todos os atributos da classe e clique em “Generate”.




```
87          return Objects.equals(id, other.id);
88      }
89
90      /**
91       * Para facilitar a identificação de um objeto em um momento de
92       * debug, precisamos de uma representação textual que nos permita
93       * identificar qual objeto é aquele, para isso vamos sobreescrever
94       * o método toString()
95       *
96       * Atalho : ALT + SHIFT + S >> Generate toString()
97       */
98      @Override
99      public String toString() {
100         return "Produto [id=" + id + ", nome=" + nome + ", descricao=" + descricao + ", preco=" + preco
101                 + ", dataCriacao=" + dataCriacao + "]";
102     }
103
104
105 }
106
```

O método **`toString`** facilita a identificação de um objeto quando estamos fazendo o *debug* da aplicação, isto é, quando estamos procurando identificar uma falha no programa. Ele vai fornecer uma descrição textual do estado do objeto quando, por exemplo, colocarmos o curso sobre a variável que possui a referência para o objeto.



16 – No Git Bash execute os comandos `git status`, `git add .`, `git commit -m "Add bean Produto"`, `git push origin main`

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        src/main/java/br/com/belval/crudrest/model/

nothing added to commit but untracked files present (use "git add" to track)

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git add .
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   src/main/java/br/com/belval/crudrest/model/Produto.java


takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git commit -m "Add bean Produto"
[main 1a6f89e] Add bean Produto
 1 file changed, 110 insertions(+)
 create mode 100644 src/main/java/br/com/belval/crudrest/model/Produto.java

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```

```
takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$ git push origin main
Enumerating objects: 19, done.
Counting objects: 100% (19/19), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (11/11), 1.60 KiB | 327.00 KiB/s, done.
Total 11 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/ProfNpc/crudrest.git
   3dbbfb6..1a6f89e  main -> main

takol@DESKTOP-8THFTQM MINGW64 /d/Aulas/Material/Spring/CrudRestExample/2026-2/workspace/crudrest (main)
$
```